



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



MODULE 4

INTERMEDIATE CODE GENERATION

Dr. B.V. Baiju,
SCOPE, Assistant Professor
VIT, Vellore

Backpatching

- Backpatching is basically a process of *filling unspecified information*.
- This information is of labels.
- It basically uses the appropriate semantic actions during the process of code generation.
- It may indicate the **address instead of label** in goto statements while producing TACs for the given expressions.
- Backpacking in compiler design refers to *reducing the size of a compiler* by *removing unnecessary components*, such as **unused variables, functions, or code**, to make it more efficient and optimized.
- Backpatching is commonly used in compilers for languages that
 - **support complex control structures**
 - **dynamic memory allocation.**

Need for Backpatching

- The syntax directed definition to generate three-address code is typically done in two passes.
 - In the **first pass** the *syntax tree* is constructed and annotated with rules.
 - In the **second pass** a *depth first search of the tree* is carried out to perform syntax directed translation.
- If a statement consists of *multiple lines of instructions* then the labels to branch may not be known in advance if SDD needs to be done in a single pass.
- To address this issue we introduce a technique called backpatching.
- The fundamental behind the technique of backpatching is
 - To generate a *series of branching statements* with the target of jumps unspecified in the **first pass**.
 - In the **second pass** we put each statement in a *list* and fill them with appropriate true and false labels.

One-Pass Code Generation Using Backpatching

- *Backpatching* is used to generate code for boolean expressions and flow control statements.
- Synthesized attributes **truelist** and **falselist** of nonterminal **B** are used to **manage labels in jumping code** for boolean expressions.
- **B.truelist** will be a list of jump or conditional jump instructions into which we must insert the label to which control goes if *B* is true.
- **B.falselist** is the list of instructions that eventually get the label to which control goes when *B* is false.
- A statement *S* has a synthesized attribute **S.nextlist**, denoting a list of jumps to the instruction immediately following the code for *S*.

Functions to incorporate backpatching

Functions	Description
makelist(i)	- This is used to create a new list containing three-address location i, and it returns a pointer to the list. - This is the first function which is created to form a true / false list.
merge(p1, p2)	- This function concatenates lists pointed to by p1 and p2, returns a pointer to the concatenated list. - Used to assign the same true / false labels to more than one address.
backpatch(p, i)	- This function is used to insert 'i' as the target label for each of the statements in the list pointed to by p. - Using the information provided by this function labels are attached to all the statements.

1. Backpatching for Boolean Expressions

- A **marker** nonterminal **M** causes a semantic action to pick up, at appropriate times, the index of the next instruction to be generated.
- nextquad()** : used to *generate the next line number* for generating three-address code and that is the reason behind introducing the new nonterminal M.

B → **B** | **B** | **B** && **B** | **!B** | (**B**) | **E** rel **E** | **true** | **false**

Semantic action for incorporating Backpatching

Production	Semantic Action
$E \rightarrow E1 \text{ or } ME2$	{ backpatch(E1 .falselist, M.quad); E.truelist := merge(E1 .truelist, E2 .truelist); E.falselist := E2 .falselist }
$E \rightarrow E1 \text{ and } M E2$	{ backpatch(E1 .truelist, M.quad); E.truelist := E2 .truelist; E.falselist := merge(E1 .falselist, E2 .falselist); }
$E \rightarrow \text{not } E1$	{ E.truelist := E1 .falselist; E.falselist := E1 .truelist }
$E \rightarrow (E1)$	{ E.truelist := E1 .truelist; E.falselist := E1 .falselist }
$E \rightarrow \text{id1 relop id2}$	{ E.truelist := makelist(nextquad()); E.falselist := makelist(nextquad() + 1); emit('if' id 1 .place relop.op id 2 .place 'goto _'); emit('goto _') }
$E \rightarrow \text{true}$	{ E.truelist := makelist(nextquad()); E.falselist := nil; emit('goto _') }
$E \rightarrow \text{false}$	{ E.falselist := makelist(nextquad()); E.truelist := nil; emit('goto _') }
$M \rightarrow \epsilon$	{ M.quad := nextquad() }

$M \rightarrow \epsilon$

- The semantic rule associated with this variable helps in generating the next line number to generate three address code

$E \rightarrow E1 \text{ or } ME2$

- Merge function concatenates the truelist of E1 and E2.
- If E1 is false we need to associate the false list of E1 with the next line number using M.quad.
- This line will contain the first instruction corresponding to E2 as this will be evaluated only if E1 is false.
- The expression E's false list will be E2's false list after incorporating short circuit

$E \rightarrow E1 \text{ and } M E2$

- As the operator is 'and', we merge the falselist of E1 and E2's and assign as E's false list.
- The true list of E is E2's true list as we will be executing E2 only if E1 is true.
- To execute E2, we backpatch E1's true to the line number corresponding to E2's first instruction which is given by M.quad

E → not E1

- The false and true lists of E and E1 are reversed.

E → (E1)

- The false and true lists of E and E1 are the same as the parenthesis is just to prioritize the expression E1

E → id1 relop id2

- The line numbers of truelist and falselist for E is considered as the next line number and its following line number.
- The code is generated using “emit” and the goto is left blank which will be backpatched later.

E → true

- Basic terminating production which will generate a goto blank which will be backpatched with truelist's number.

E → false

- Basic terminating production which will generate a goto blank which will be backpatched with falselist's number.

- Consider the expression

`x < 100 || y > 200 && x != y`

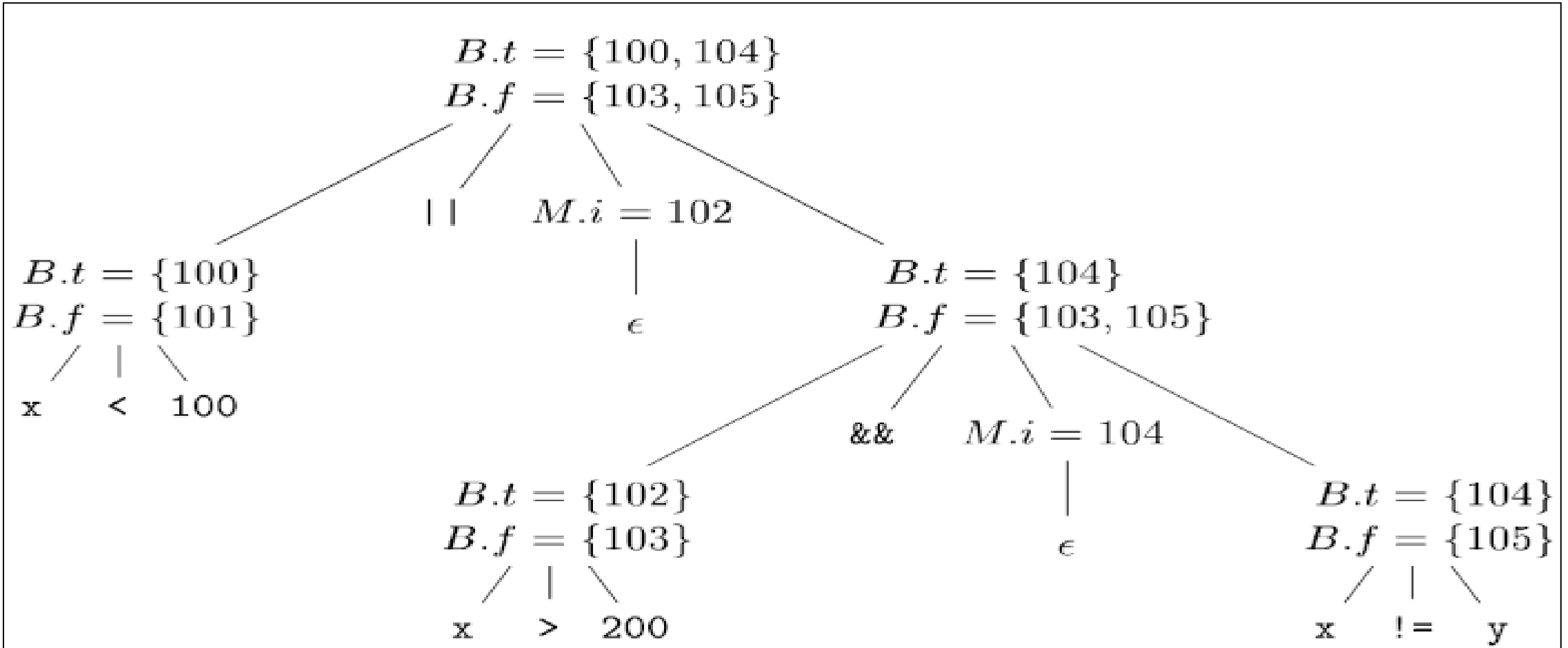
First pass where to go next is undefined

```
100: if x < 100 goto ____  
101: goto ____  
102: if y > 200 goto ____  
103: goto ____  
104: if x!=y goto ____  
105: goto ____  
106: true  
107: false
```

Second pass using backpatching

```
100: if x < 100 goto 106  
101: goto 102  
102: if y > 200 goto 104  
103: goto 107  
104: if x!=y goto 106  
105: goto 107  
106: true  
107: false
```

Annotated parse tree



- Attributes **truelists** and **falselists** are represented by **t** and **f** respectively.
- **Actions** are performed during a *depth first search traversal of the tree* and since they all appear at the ends of the right sides, they are performed together with reductions during bottom-up parsing.

2. Flow-of-Control Statements

- Backpatching is used to translate flow-of-control statements in one pass.

$\begin{aligned} S &\rightarrow \text{if (B) } S \mid \text{if (B) } S \text{ else } S \mid \text{while (B) } S \mid \{L\} \mid A \\ L &\rightarrow LS \mid S \end{aligned}$
--

S denotes a *statement*

L a *list*

A an *argument*

B a *boolean*

- The attributes of S is **S.nextlist** which will backpatch list for jumps to the next statement after S.
- The attributes of L is **L.nextlist** which will backpatch list for jumps to the next statement after L

Semantic rules for control flow statements to incorporate backpatching

Production	Semantic Action
$S \rightarrow A$	{ S.nextlist := nil }
$S \rightarrow \text{begin } L \text{ end}$	{ S.nextlist := L.nextlist }
$S \rightarrow \text{if } E \text{ then } M \text{ } S1$	{ backpatch(E.truelist, M.quad); S.nextlist := merge(E.falselist, S1 .nextlist) }
$S \rightarrow \text{if } E \text{ then } M \text{ } S1 \text{ } N$ else M2 S2	{ backpatch(E.truelist, M1 .quad); backpatch(E.falselist, M2 .quad); S.nextlist := merge(S1 .nextlist, merge(N.nextlist, S2 .nextlist)) }
$S \rightarrow \text{while } M1 \text{ } E \text{ do } M2$ S1	{ backpatch(S1 .nextlist, M1 .quad); backpatch(E.truelist, M2 .quad); S.nextlist := E.falselist; emit('goto _') }
$L \rightarrow L1 ; M \text{ } S$	{ backpatch(L1 .nextlist, M.quad); L.nextlist := S.nextlist; }
$L \rightarrow S$	{ L.nextlist := S.nextlist; }
$N \rightarrow \epsilon$	{ N.nextlist := makelist(nextquad()); emit('goto _') }

S → A

- This production is a termination production and hence there is no need for a backpatch

S → begin L end

- Both S and L has a nextlist attribute and they are set to the same.
- The statements between 'begin' and 'end' are run only once.

S → if E then M S1

- The variable M produces ϵ .
- If the expression is false, then the statement S1 need to be skipped.
- If expression is true, then S1 should be executed.
- In both the scenarios, the statement that is available outside S1 need to be continued.
- To carry out this, the falselist of E and the nextlist of S1 are merged and that is assigned as S's nextlist.
- If expression is true then the statement S1 is to be executed.
- To incorporate this we backpatch the truelist of the expression to S1 which is done with the help of M.quad

N → ϵ

- The goto is to skip S2 which is part of if-then-else and go to the statement following S2.

S → if E then M S1 N else M2 S2

- The expression is evaluated and if it is true S1 is to be executed and S2 if it is false S1 is to be executed.
- This is implemented by backpatching the truelist and falselist to M1.quad and M2.quad which is the beginning of statements S1, S2 respectively.
- After executing S1, S2 need to be skipped and the statement which is available after S needs to be executed.
- After executing S2 by skipping S1, we need to execute the statement outside the body of S. To incorporate this we use the symbol N to skip S2.
- We assign the nextlist of S as S1's next and S2's next

S → while M1 E do M2 S1

- The variable M2 helps to go to go to statement S1 if the expression is true.
- If the expression is false then we need to go to the statement following S1 which is done as the nextlist of S the same as E's falselist.
- To incorporate continuation of the loop, M1 is used which is to come back after finishing S1.
- The goto is to loop again to execute the expression E

$S \rightarrow \text{while } M_1 \text{ E do } M_2 S_1$

```
backpatch( $S_1.\text{nextlist}$ ,  $M_1.\text{addr}$ );  
backpatch( $E.\text{truelist}$ ,  $M_2.\text{addr}$ );  
 $S.\text{nextlist} := E.\text{falselist}$ ;  
emit("goto  $M_1.\text{addr}$ ");
```

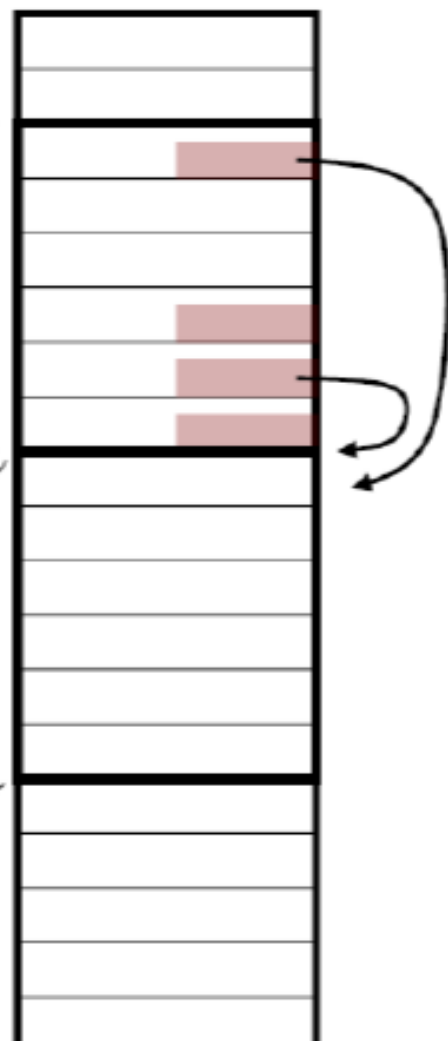
E

```
100: if a < b goto _  
101: goto 102  
102: if c < d goto 104  
103: goto _  
104: if e < f goto _  
105: goto _
```

$E.\text{truelist} = \{100, 104\}$

$E.\text{falselist} = \{103, 105\}$

S_1



Acti
Go to

$S \rightarrow \text{while } M_1 \text{ E do } M_2 S_1$

```
backpatch(S1.nextlist, M1.addr);
backpatch(E.truelist, M2.addr);
S.nextlist := E.falselist;
emit("goto M1.addr");
```

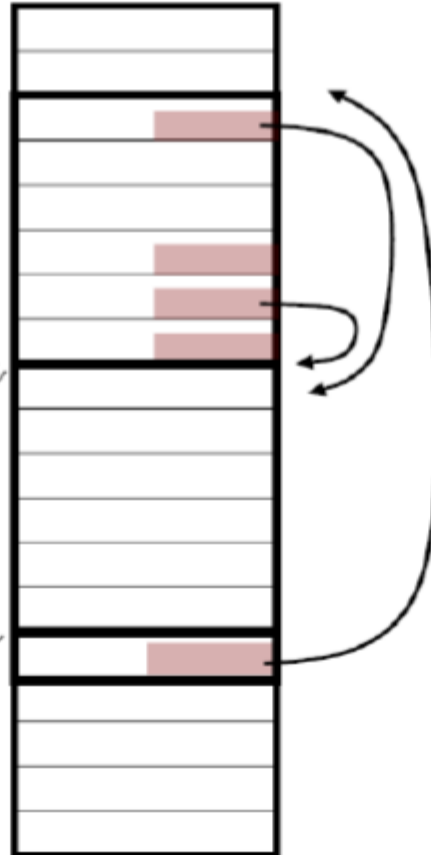
E

```
100: if a < b goto _
101: goto 102
102: if c < d goto 104
103: goto _
104: if e < f goto _
105: goto _
```

E.truelist = {100, 104}

E.falselist = {103, 105}

S₁



Acti
Go to

7. Generate three address code for the following code

```
prod = 0 ;  
i = 1 ;  
do  
{  
  prod = prod + a[ i ] x b[ i ] ;  
  i = i + 1 ;  
} while (i <= 10) ;
```

The three-address code for the given program segment is given below:

```
1. prod = 0  
2. i = 1  
3. t1 = 4 x i  
4. t2 = a[t1]  
5. t3 = 4 x i  
6. t4 = b[t3]  
7. t5 = t2 x t4  
8. t6 = t5 + prod  
9. prod = T6  
10. t7 = i + 1  
11. i = t7  
12. if (i <= 10) goto (3)
```

Address & Pointers:

x = &y;

*a = b;

p = *q;

Intermediate Code:

100: t₁ = &y

101: x = t₁

102: t₂ = *a

103: t₂ = b

104: t₃ = *q

105: p = t₃

op	arg1	arg2	result
&	y	-	t ₁
=	t ₁	-	x
*	a	-	t ₂
=	b	-	t ₂
*	q	-	t ₃
=	t ₃	-	p

Quadruples

	op	arg1	arg2
0	&	y	-
1	=	x	(0)
2	*	a	-
3	=	(2)	b
4	*	q	-
5	=	p	(4)

Triples

Program Segment:

```
int fact(int n)
{ if(n == 0) return 1;
  else return (n * fact(n - 1));
}
```

Intermediate Code:

```
100:    function begin fact
101:    if (n == 0) goto 108
102:    t1 = n - 1
103:    param t1
104:    refparam res
105:    call fact, 2
106:    t3 = n * res
107:    return t3
108:    return 1
109:    function end
110:    exit
```

Program Segment:

```
do
{
    i = i + 1;
} while(a[i] < x);
```

Intermediate Code:

```
L:      t1 = i + 1
        i = t1
        t2 = i * 4
        t3 = a[t2]
        if t3 < x goto L
```

Intermediate Code:

```
100:    t1 = i + 1
101:    i = t1
102:    t2 = i * 4
103:    t3 = a[t2]
104:    if t3 < x goto 100
```

op	arg1	arg2	result
+	i	1	t ₁
=	t ₁	-	i
*	i	4	t ₂
= []	a	t ₂	t ₃
<	t ₃	x	t ₄
if	t ₄	L	-

Quadruples

	op	arg1	arg2
0	+	i	1
1	=	i	(0)
2	*	i	4
3	= []	a	(2)
4	<	(3)	x
5	if	(4)	L

Triples